CS433: Assignment#1 (Due by 30th September 2026 midnight)

1. In this problem, you will write a shared memory parallel program to find out the minimum number of colors needed to color the edges and the vertices of an undirected graph such that (i) the end-point vertices u and v of an edge (u, v) receive different colors, (ii) the edges incident on a vertex w receive colors different from the color of w, and (iii) each triangle receives distinct colors on its edges. A triangle is a subgraph having three vertices u, v, w and three edges $(u, v), (v, w), (w, u)$. The program will accept an input file name, an output file name, and the number of threads from the command line exactly in that order. The input file describes the graph. I have included a sample input file so that you can generate similar files for testing. The first line in the file contains the number of vertices in the graph. The next line contains the list of vertices connecting vertex#0. In general, a particular line lists the vertices i connecting vertex#k such that $i > k$. The sample file describes a complete graph of four vertices. The second line shows the edges (0, 1), (0, 2), (0, 3). The next line shows the edges (1, 2), (1, 3). The last line shows the edge (2, 3). It is very important to adhere to the input file format because we will test your submission on graphs generated by us. So, even if you have written a correct and efficient parallel program, if we cannot run your program on our graphs, we won't be able to give you any score. When you generate a graph of your own, ensure that the graph has some triangles. One way to ensure this is to pick random triplets of vertices and connect them by edges. The program should output the color assignment and the minimum number of colors needed to the output file in the following format. For a graph with n vertices and m edges, the first $m + n$ lines of the output file should list the color assignment using the notation `u, c` or `(u, v), c` where `u` is a vertex with color `c` or `(u, v)` is an edge with color `c`. The colors should be given integer ids starting from 0. The last line of the output file should list the minimum number of colors needed. Note that depending on the graph structure, it is possible to have multiple different color assignments for the same minimum number of colors. It is sufficient to output any one of them. Please feel free to choose OpenMP or POSIX thread for your implementation. **(10 points)**

2. In this problem, you will write a shared memory parallel program to solve a system of equations $Ax = y$ where A is a $(2n + 1) \times (2n + 1)$ matrix of reals and x, y are real-valued $(2n + 1) \times 1$ vectors. The problem is to find x given A and y. The matrix A has a specific structure as discussed next. Suppose the rows and the columns of A are numbered from 0 to $2n$. Row i of A has non-zero values only between columns i and $2n - i$ (both inclusive) and zeros elsewhere. The program will take an input file name, an output file name, and the number of threads from the command line exactly in that order. The first line of the input file specifies the value of n. The second line onward it lists the non-zero elements of A row-wise. The last line of the file lists the elements of y. I have included a sample file specifying the following A and y.

$$A = \begin{bmatrix} 1.0 & -2.0 & 6.2 \\ 0 & 2.2 & 0 \\ 3.0 & 4.1 & -1.3 \end{bmatrix}, \quad y = \begin{bmatrix} 2.3 \\ 4.5 \\ -7.2 \end{bmatrix}.$$

Note that reading large matrices from the file may take a significant amount of time. So, for testing your program on large matrices quickly, you can initialize A and y with arbitrary values in your program. Please put this code in a clearly marked function named InitializeInput. Comment out the call to this function in your submission so that we can test your program with our input test files. The program should output the elements of x in the first line of the output file. The output file should not have any other lines. Please feel free to choose OpenMP or POSIX thread for your implementation. Hint: Do not invert A and then multiply by y. Use simpler ways of finding x. **(10 points)**

What to submit and how. For each question, submit the parallel program. Prepare a report describing your parallel algorithms, the optimizations you applied (if any) for improving performance, and the performance results. Please clearly specify in the report whether your program is written using OpenMP directives or POSIX thread libraries for each program.

For the first question, report results for at least five graphs. For each graph, prepare a table showing how the measured time varies with the thread count (the range of thread count is left to you). Try to pick largish graphs that your program can run within reasonable time. Remember to specify the number of vertices and edges in each graph clearly. Discuss the observed trends with explanation.

For the second question, report results for at least five values of n. For each n, prepare a table showing how the measured time varies with the thread count. Remember to specify the values of n clearly. Try to pick large n that your program can run within reasonable time. Discuss the observed trends with explanation. To generate a numerically stable test case, please generate A and x first, multiply them to get y, and use A and y as inputs to your program. This way you will also know what the correct x is. You need to ensure that none of the elements of A is too small or too large in magnitude. This is just a suggestion. Of course, you may use different ways of generating test cases.

Note that your marks will depend on how good your program is in terms of performance, how well the algorithm and the program are described in the report, and the discussion on the results. In your measured time, do not include any time that is needed to allocate memory or initialize arrays. Please refer to the examples discussed in the class. You will not get any marks if your program does not compile or crashes during execution. Only the programs that compile and run to completion without any error will be considered for grading. Please do not use any external library outside POSIX threads, OpenMP, and standard C/C++. That will make testing your code very difficult. Please feel free to refer to and cite papers/books, but do not copy codes from other places.

Please send your submission (two programs and a PDF report) via email to mainakc@cse.iitk.ac.in with subject "[CS433] Assignment#1 submission Group#X". Replace X by your group number in the subject line.